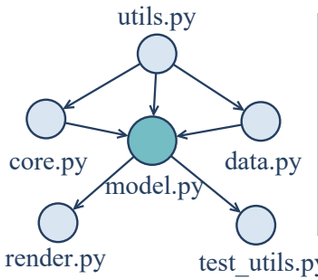
Information sources

Requirements

Contracts

Human judgments




```
def
process(x):
  ...

  ...
```

Code / dependencies

```
def test_sort():
  ...
def test_edge():
  ...
```

Existing tests / traces

```
> test_sort
> ...
> x = [3,1,2]
> y = [1,2,3]
> assert ...
```

trace.log

Reference

```
def sort(x):
  return (
    sorted(x))
```

Candidate

```
def sort(x):
  ...
```

Reference agreement is conditional

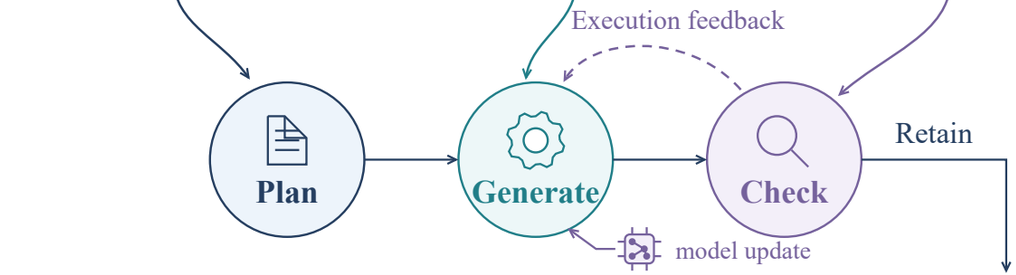
Test construction

Method families (alternative or combinable)

Prompting + retrieval

Program analysis + search

Agent orchestration



Constructed test (illustrative)

Setup	Inputs / actions	Oracle
`before = copy(x)`	`y = sort(x)`	`assert ordered(y)` `assert x == before`

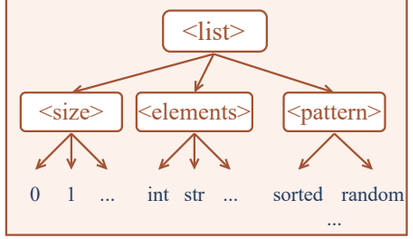
Scenarios

```
# reverse order
x = [3, 2, 1]
...
```

Executable tests

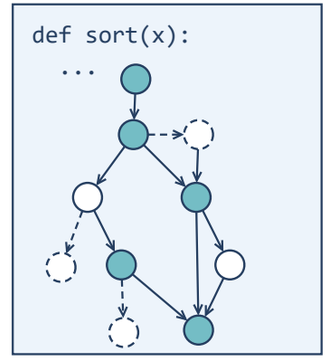
```
setup: before = copy(x)
run: y = sort(x)
assert: ordered(y)
assert: x == before
```

Generators / mutators



Quality evidence

Operational evidence



- Execution
- Coverage
- Stability / cost

- Executed path
- Uncovered path

Behavioral evidence

Specification

```
Preconditions:
  ...
Postconditions:
  ...
Invariants:
  ...
```

Valid programs (legal behavior)

```
def sort(x):
  ...
```

Faulty programs (incorrect behavior)

```
def sort(x):
  ...
```

Independent evaluation

- Oracle validity
- Accept legal behavior
- Detect faulty behavior

Learning across tasks

Training-time information

Training data
Code, tests, specifications, human feedback, ...

SFT / RL / self-play
Learn generation policy across tasks

Updated generator
Better priors for test generation

Unit / regression · Requirements / acceptance · API / GUI · Fuzzing · Bug reproduction